

PRACTICAL WEEK 4: 2520090058

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions

ANS:

Program using `wait()`;

CODE:

```
1. UN PICO 3.09 File: q.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i, status;
    pid_t pid;

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            // Child process
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i, getpid(), getpid());

            sleep(i); // Different execution times
            printf("Child %d completed\n", i);
            exit(i * 10);
        }
    }

    // Parent waits for all children
    for (i = 1; i <= 3; i++)
    {
        pid = wait(&status);

        if (WIFEXITED(status))
        {
            printf("Parent: Child PID %d terminated with status %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("Parent process completed.\n");

    return 0;
}
```

OUTPUT:

```
Last login: Sat Sep 12 08:13:24 on console
screen@hillelreddey@Stress-MacBook-Pro: ~ % nano q.c
screen@hillelreddey@Stress-MacBook-Pro: ~ % gcc q.c -o q
screen@hillelreddey@Stress-MacBook-Pro: ~ % ./q
Parent process started. PID = 1088
Child 1: PID = 1088, Parent PID = 1088
Child 2: PID = 1089, Parent PID = 1088
Child 3: PID = 1090, Parent PID = 1088
Child 3 completed
Parent: Child PID 1088 terminated with status 10
Child 2 completed
Parent: Child PID 1089 terminated with status 20
Child 3 completed
Parent: Child PID 1090 terminated with status 30
Parent process completed.
screen@hillelreddey@Stress-MacBook-Pro: ~ %
```

PROGRAM USING `waitpid()`;

CODE:

```
UN FICD 5.09 File: bb.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i, status;
    pid_t pid;
    pid_t child[3];

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (i = 0; i < 3; i++)
    {
        pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }
        if (pid == 0)
        {
            printf("Child %d started. PID = %d\n",
                   i + 1, getpid());

            sleep(3 - i);

            printf("Child %d completed.\n", i + 1);
            exit((i + 1) * 10);
        }
        // Parent stores child PID
        child[i] = pid;
    }

    // Wait for each specific child
    for (i = 0; i < 3; i++)
    {
        pid = waitpid(child[i], &status, 0);

        if (pid > 0 && WIFEXITED(status))
        {
            printf("Parent collected child PID %d, Exit status = %d\n",
                   pid, WEXITSTATUS(status));
        }
    }

    printf("All children completed. Parent exiting.\n");
    return 0;
}
```

OUTPUT:

```
screen1@hilleddy@rees-MacBook-Pro ~ % nano bb.c
screen1@hilleddy@rees-MacBook-Pro ~ % gcc bb.c -o bb
screen1@hilleddy@rees-MacBook-Pro ~ % ./bb
Parent process started. PID = 1185
Child 1 started. PID = 1186
Child 2 started. PID = 1187
Child 3 started. PID = 1188
Child 3 completed.
Child 2 completed.
Child 1 completed.
Parent collected child PID 1186, Exit status = 10
Parent collected child PID 1187, Exit status = 20
Parent collected child PID 1188, Exit status = 30
All children completed. Parent exiting.
screen1@hilleddy@rees-MacBook-Pro ~ %
```

Create a scenario where a child process becomes a zombie process.
Investigate the process table and then modify the program to
eliminate zombie processes using proper synchronization techniques.

C PROGRAM

CODE:

```
1 1m FICD 5.09 File: v.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started\n");
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");
        wait();
    }
    else
    {
        // Parent process
        printf("Parent process started\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        // Parent does NOT call wait()
        printf("Parent sleeping for 20 seconds...\n");
        sleep(20);

        printf("Parent process terminating...\n");
    }

    return 0;
}
```

OUTPUT:

```
screen@hillelreddy@Stress-MacBook-Pro ~ % nano v.c
screen@hillelreddy@Stress-MacBook-Pro ~ % gcc v.c -o v
screen@hillelreddy@Stress-MacBook-Pro ~ % ./v
Parent process started
Parent PID = 1209
Child PID = 1218
Parent sleeping for 20 seconds...
Child process started
Child PID = 1218
Child process terminating...
nano.v.c
Parent process terminating...
screen@hillelreddy@Stress-MacBook-Pro ~ % nano.v.c
ssh: command not found: nano.v.c
screen@hillelreddy@Stress-MacBook-Pro ~ % nano v.c
screen@hillelreddy@Stress-MacBook-Pro ~ % █
```

Eliminate using the zombie wait()

```
UN FICD 5.09 File: bh.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started\n");
        printf("Child PID = %d\n", getpid());
        sleep(2);

        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("Parent process started\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        // Parent waits for child
        wait(&status);

        if (WIFEXITED(status))
        {
            printf("Child terminated normally\n");
            printf("Child exit status = %d\n",
                    WEXITSTATUS(status));
        }

        printf("No zombie process remains\n");
    }

    return 0;
}
```

OUTPUT:

```
streeni@hilleddy@Stress-MacBook-Pro ~ % nano bh.c
streeni@hilleddy@Stress-MacBook-Pro ~ % gcc bh.c -o
clang: error: argument to '-o' is missing (expected 1 value)
streeni@hilleddy@Stress-MacBook-Pro ~ % gcc bh.c -o bh
streeni@hilleddy@Stress-MacBook-Pro ~ % ./bh
Parent process started
Parent PID = 1242
Child PID = 1243
Child process started
Child PID = 1243
Child process terminating...
Child terminated normally
Child exit status = 0
No zombie process remains
streeni@hilleddy@Stress-MacBook-Pro ~ % nano bh.c
streeni@hilleddy@Stress-MacBook-Pro ~ % █
```